

A Comprehensive Measure of Well-Being

Project Description:

The Human Development Index (HDI) is a statistical composite index of life expectancy, education, and per capita income indicators, which are used to rank countries into four tiers: Very High, High, Medium, and Low of human development. A country achieves a higher HDI when its population enjoys a longer lifespan, better educational attainment, and a higher Gross National Income (GNI PPP) per capita.

The HDI was developed to emphasize that people and their capabilities should be the primary measure of a country's development rather than economic growth alone. It provides a broader perspective on development by considering health, education, and living standards together. The index is widely used by governments, researchers, policymakers, and international organizations to evaluate progress, compare countries, and identify areas requiring improvement.

Scenarios

Scenario 1: Predicting Very High Human Development

A user selects a country with high life expectancy, strong mean years of schooling, expected years of schooling, and a high GNI per capita. Based on these indicators, the model predicts a Very High HDI score, classifying the country among the most developed nations. This helps users understand how strong performance across multiple development dimensions contributes to overall human development.

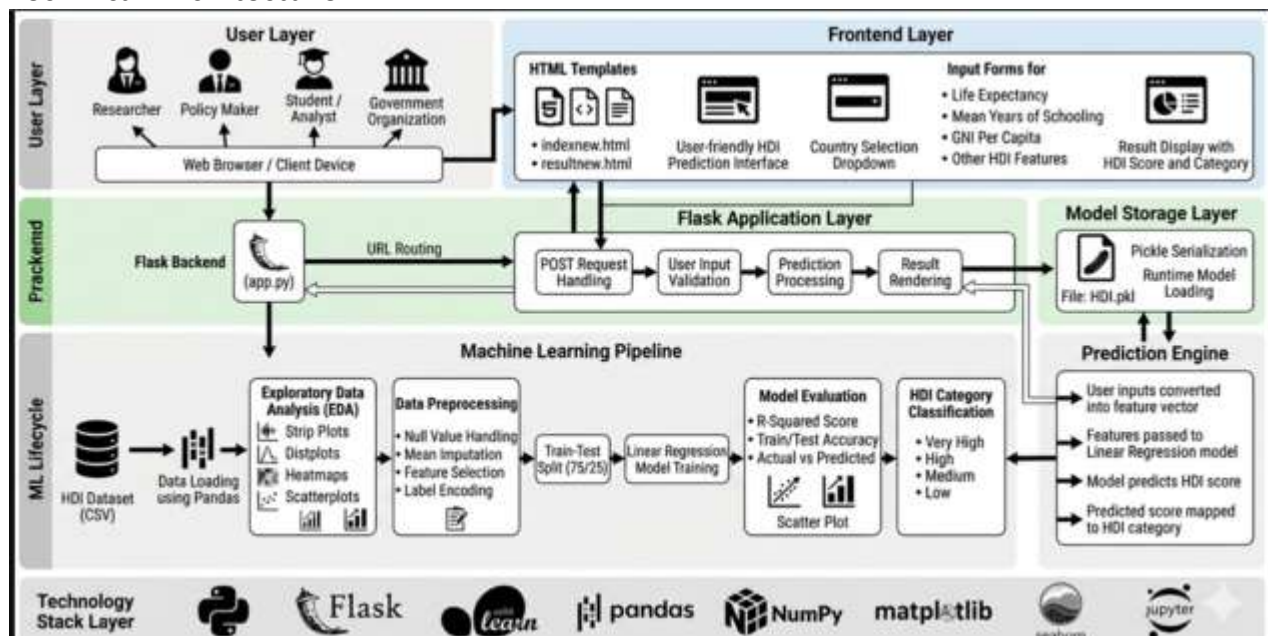
Scenario 2: Identifying Development Gaps in Emerging Economies

A policymaker inputs mid-range values representing a developing nation, including moderate life expectancy, average educational attainment, and a moderate income level. The model returns a Medium HDI score, providing insights into areas where improvements in healthcare, education, or income generation could significantly enhance human development outcomes.

Scenario 3: Assessing Countries Requiring Development Intervention

A researcher evaluates a country with relatively low life expectancy, limited educational opportunities, and low GNI per capita. The model predicts a Low HDI score and highlights the country's developmental challenges. Such predictions can support governments and development organizations in prioritizing investments and policy interventions.

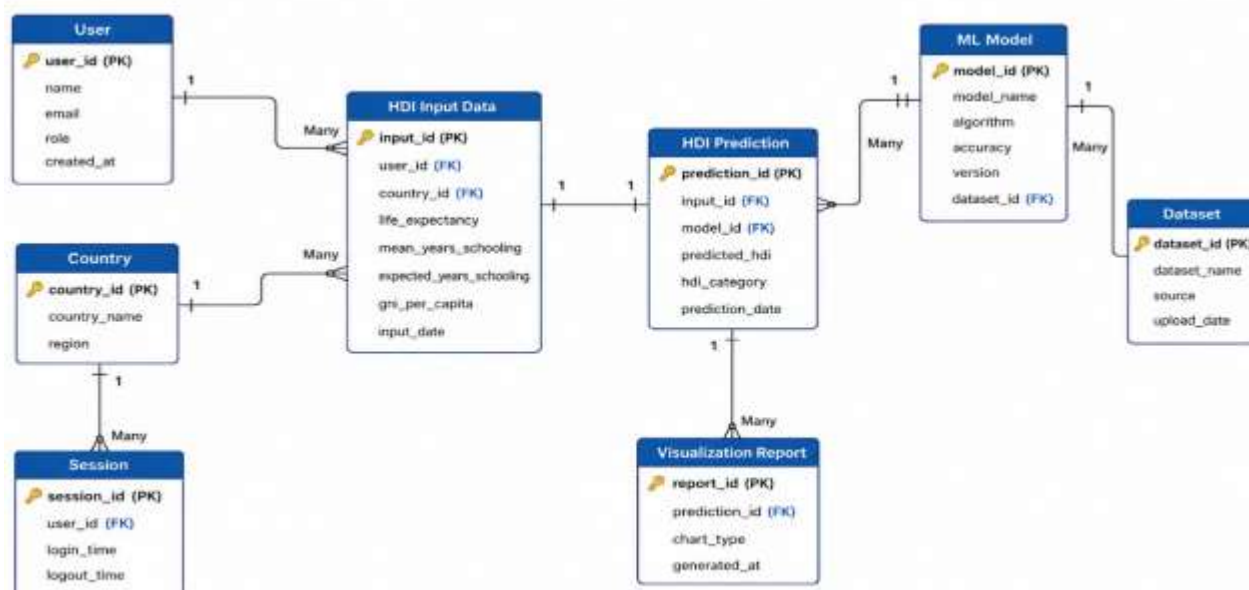
Technical Architecture:



Description:

The architecture diagram represents the complete workflow of the **Human Development Index (HDI) Prediction System**. It consists of the **User Layer**, **Frontend Layer**, **Flask Application Layer**, **Machine Learning Pipeline**, **Model Storage Layer**, **Prediction Engine**, and **Technology Stack Layer**. Users provide HDI-related inputs through a web interface, which are processed by the Flask application. The machine learning pipeline performs data preprocessing, model training, and prediction using a **Linear Regression** model. The trained model is stored as a Pickle file and loaded during runtime for efficient predictions. Finally, the predicted HDI score and its corresponding development category are displayed to the user through the web application.

Entity Relationship Diagram:



Description:

The Entity Relationship Diagram (ERD) illustrates the logical structure of the Human Development Index (HDI) Prediction System and the relationships between its entities. It includes the **User**, **Country**, **HDI Input Data**, **HDI Prediction**, **Machine Learning Model**, **Dataset**, **Session**, and **Visualization Report** entities. Users submit HDI-related input data for a selected country, which is processed by the trained machine learning model to generate HDI predictions. The prediction results are stored along with the model and dataset information for traceability. User sessions and visualization reports help monitor system usage and present prediction results effectively. This ERD provides a clear representation of data organization, entity relationships, and information flow within the HDI prediction system.

Pre-requisites:

Python: <https://www.python.org/downloads/>

Jupyter Notebook: <https://jupyter.org/install/>

Visual Studio Code: <https://code.visualstudio.com/>

NumPy: <https://numpy.org/doc/stable/>

Pandas: <https://pandas.pydata.org/docs/>

Scikit-learn: <https://scikit-learn.org/stable/>

Matplotlib: <https://matplotlib.org/stable/>

Seaborn: <https://seaborn.pydata.org/>

Flask: <https://flask.palletsprojects.com/>

Git: <https://git-scm.com/downloads>

GitHub: <https://github.com/>

Render: <https://render.com/>

Kaggle (Human Development Index Dataset: <https://www.kaggle.com/>)

Project Workflow:

The project is developed through a structured workflow consisting of multiple epics, covering data collection, analysis, preprocessing, model development, and deployment. Each epic focuses on a specific stage of the machine learning lifecycle to ensure systematic and efficient project execution.

Epic 1: Environment Setup and Package Installation

Story 1: Download and install all required Python packages, machine learning libraries, and Flask dependencies needed for the project.

Story 2: Create and organize the project folder structure, including directories for datasets, models, templates, static files, and source code.

Epic 2: Importing Required Libraries

Story 1: Import all necessary libraries and modules such as NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, Pickle, and Flask to support data analysis, model development, and web application creation.

Epic 3: Dataset Download and Understanding

Story 1: Download the dataset from Kaggle and load it into the development environment.

Story 2: Explore and understand the dataset structure, features, target variable, and data types.

Story 3: Perform data visualization to identify patterns, trends, distributions, and relationships within the dataset.

Epic 4: Data Preprocessing and Label Encoding

Story 1: Select dependent and independent variables required for model training.

Story 2: Check for missing values and handle null entries appropriately to improve data quality.

Story 3: Perform label encoding to convert categorical text values into numerical representations suitable for machine learning algorithms.

Story 4: Prepare the cleaned and transformed dataset for model development.

Epic 5: Dividing the Dataset into Train and Test Data

Story 1: Split the processed dataset into training and testing sets to enable model learning and performance evaluation on unseen data.

Epic 6: Fitting the Model

Story 1: Train the Linear Regression model using the prepared training dataset.

Story 2: Generate predictions using the trained model and analyze the prediction results.

Story 3: Evaluate model performance using appropriate regression metrics and visualizations.

Epic 7: Saving the Model

Story 1: Save the trained Linear Regression model using serialization techniques such as Pickle so it can be reused without retraining.

Story 2: Store the model file for deployment and future prediction purposes.

Epic 8: Building the Flask Web Application

Story 1: Develop the Flask backend to handle user requests, process inputs, load the trained model, and generate predictions.

Story 2: Create HTML templates and integrate them with Flask to provide a user-friendly interface.

Story 3: Run, test, and validate the complete web application to ensure accurate predictions and smooth functionality.

Epic 1: Environment Setup and Package Installation

Story 1: Download Required Packages:

pip install numpy: Used for numerical operations, array handling, and mathematical computations required in data analysis and machine learning.

pip install pandas: Used for data manipulation, preprocessing, cleaning, and handling datasets using DataFrames.

pip install Matplotlib: Used for data visualization and plotting graphs, charts, and analytical figures.

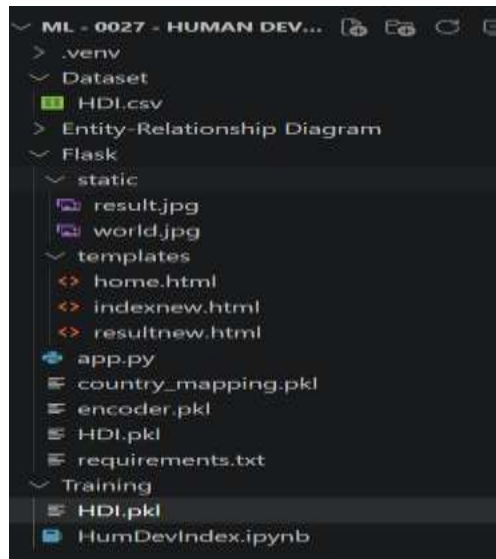
pip install scikit-learn: Used for implementing machine learning algorithms, data preprocessing, model training, and evaluation.

pip install Flask: Used for building the backend web application, handling routing, and integrating the trained machine learning model with the user interface.

```
>>> pip install pandas
>>>
>>> pip install matplotlib
>>>
>>> pip install scikit-learn
>>>
>>> pip install Flask
Requirement already satisfied: numpy in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (2.2.6)
[notice] A new release of pip available: 22.0.1 -> 23.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
Requirement already satisfied: pandas in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (2.0.3)
Requirement already satisfied: tzdata>=2022.7 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (pandas) (2023.2)
Requirement already satisfied: pytz>=2020.1 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (pandas) (2023.2)
Requirement already satisfied: numpy>=1.22.4 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (pandas) (2.2.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\hp\appdata\roaming\python\python310\lib\site-packages (pandas) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\hp\appdata\roaming\python\python310\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
[notice] A new release of pip available: 22.0.1 -> 23.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
Collecting matplotlib
  Downloading matplotlib-3.10.0-cp310-cp310-win_msvc90.whl (8.2 MB)
```

Story 2: Set Up Folder Structure:

Create the complete project folder structure as outlined in the Project Structure section. Carefully organize all directories and subdirectories according to the specified hierarchy to ensure a clean, maintainable, and scalable project setup. Once the folder structure has been created, place each file in its designated location within the appropriate directory. Verifying the correct placement of files at this stage is important, as it helps avoid path-related issues, simplifies project navigation, and ensures that all modules, resources, configurations, and assets can be accessed correctly during development and execution. After confirming that the directory structure and file organization match the prescribed project architecture, you can proceed with the remaining implementation.



Epic 2: Importing Required Libraries

Story 1: Required Libraries Install and Import:

Before building the HDI prediction model, the required Python libraries are imported to support data processing, visualization, machine learning, and web application development. This ensures an organized workflow and prepares the environment for efficient model training and prediction.

- NumPy: Core library for numerical computing and array operations.
- Pandas: Used to load, explore, and manipulate the CSV dataset.
- Matplotlib: Provides the base plotting engine for all visualizations.
- Seaborn: Used for statistical visualizations including strip plots and heatmaps.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

import pickle

%matplotlib inline
```

✓ 42.0s

Epic 3: Dataset Download and Understanding

Story 1: Reading the Dataset

Description

You can collect datasets from open sources such as Kaggle, data.gov, or the UCI Machine Learning Repository. The dataset for this project was sourced from Kaggle.

```
data = pd.read_csv("../Dataset/HDI.csv")
data.head()
```

0.3s

Story 2: Understanding the Dataset

Load the dataset into a Pandas DataFrame and inspect its structure. The dataset is provided in CSV format and contains various human development indicators for different countries.

The head() method is used to display the first five rows of the dataset, providing a quick overview of the available features and their corresponding values. The dataset consists of 195 rows and 880 columns, where each row represents a country and each column represents a specific human development metric.

data.head()

0.3s

	ISO3	Country	Human Development Groups	UNDP Developing Regions	HDI Rank (2021)	Human Development Index (1990)	Human Development Index (1995)
0	AFG	Afghanistan	Low	SA	180.0	0.273	0.273
1	AGO	Angola	Medium	SSA	145.0	NaN	NaN
2	ALB	Albania	High	ECA	67.0	0.647	0.647
3	AND	Andorra	Very High	NaN	40.0	NaN	NaN
4	ARE	United Arab Emirates	Very High	AS	26.0	0.728	0.728

5 rows x 880 columns

Story 3: Data Visualization

We perform ten visualization tasks to understand the relationships between HDI and key input features. All visualizations use the first 20 rows (data1) to avoid overcrowding in plots.

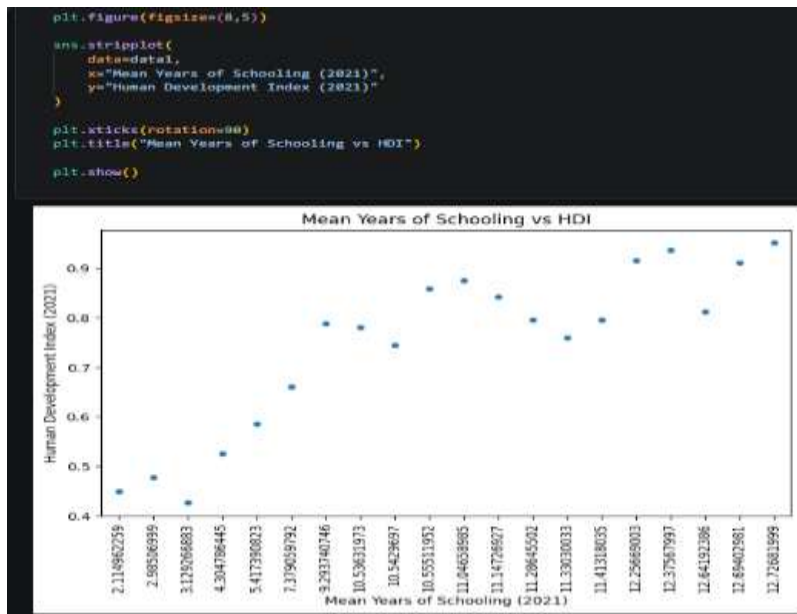
Unique Country Values: Use the unique() method on the Country column to list all unique country names, confirming there are no duplicate country entries.

```
print(data["Country"].unique())
```

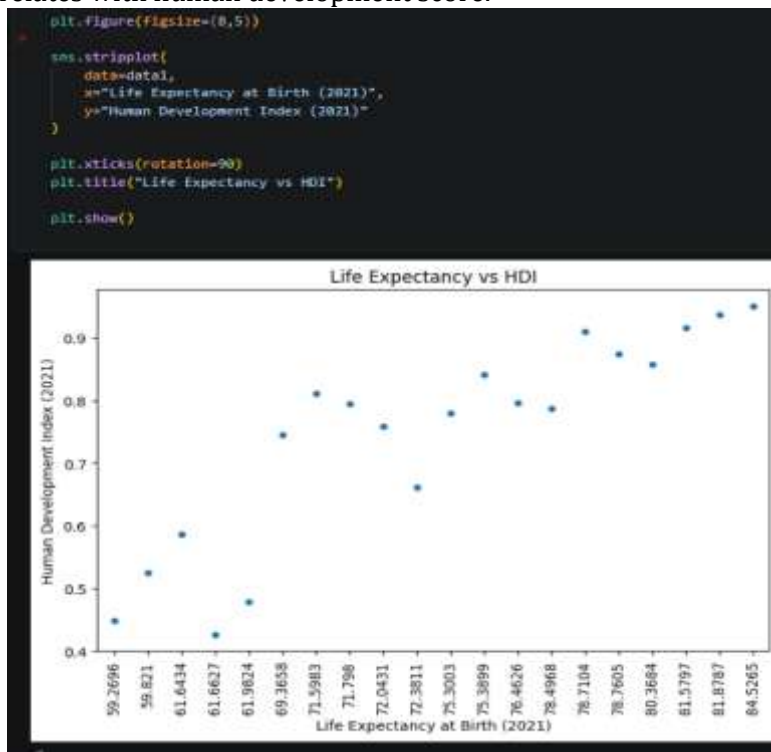
Python

```
(195, 880)
<class 'pandas.DataFrame'>
RangeIndex: 195 entries, 0 to 194
Columns: 880 entries, ISO3 to Material Footprint per capita (tonnes) (2021)
dtypes: float64(876), str(4)
memory usage: 1.3 MB
<StringArray>
[
    'Afghanistan',
    'Albania',
    'Angola',
    'Andorra',
    'Argentina',
    'Armenia',
    'Antigua and Barbuda',
    'Australia',
    'Austria',
    'Azerbaijan',
    'Bahrain',
    'Bangladesh',
    'Barbados',
    'Belarus',
    'Belgium',
    'Belize',
    'Benin',
    'Bhutan',
    'Bolivia',
    'Bosnia and Herzegovina',
    'Botswana',
    'Brazil',
    'Bulgaria',
    'Burkina Faso',
    'Burundi',
    'Cabo Verde',
    'Cambodia',
    'Cameroon',
    'Canada',
    'Central African Republic',
    'Chad',
    'Chile',
    'China',
    'Colombia',
    'Comoros',
    'Congo',
    'Costa Rica',
    'Cote d'Ivoire',
    'Croatia',
    'Cuba',
    'Cyprus',
    'Czechia',
    'Democratic Republic of the Congo',
    'Denmark',
    'Dominica',
    'Dominican Republic',
    'Ecuador',
    'Egypt',
    'El Salvador',
    'Equatorial Guinea',
    'Eritrea',
    'Estonia',
    'Eswatini',
    'Ethiopia',
    'Finland',
    'France',
    'Gabon',
    'Gambia',
    'Germany',
    'Ghana',
    'Greece',
    'Guatemala',
    'Guinea',
    'Guinea-Bissau',
    'Guyana',
    'Honduras',
    'Hungary',
    'Iceland',
    'India',
    'Indonesia',
    'Israel',
    'Italy',
    'Jamaica',
    'Japan',
    'Jordan',
    'Kazakhstan',
    'Kenya',
    'Kiribati',
    'Korea',
    'Kuwait',
    'Kyrgyzstan',
    'Laos',
    'Latvia',
    'Lebanon',
    'Lesotho',
    'Liberia',
    'Lithuania',
    'Luxembourg',
    'Madagascar',
    'Malawi',
    'Malaysia',
    'Maldives',
    'Mali',
    'Malta',
    'Mauritius',
    'Mexico',
    'Moldova',
    'Mongolia',
    'Montenegro',
    'Morocco',
    'Mozambique',
    'Myanmar',
    'Namibia',
    'Nauru',
    'Nepal',
    'Netherlands',
    'New Zealand',
    'Nicaragua',
    'Niger',
    'Nigeria',
    'North Macedonia',
    'Norway',
    'Oman',
    'Pakistan',
    'Palau',
    'Panama',
    'Papua New Guinea',
    'Paraguay',
    'Peru',
    'Philippines',
    'Poland',
    'Portugal',
    'Qatar',
    'Romania',
    'Russia',
    'Rwanda',
    'Saint Kitts and Nevis',
    'Saint Vincent and the Grenadines',
    'Samoa',
    'San Marino',
    'Sao Tome and Principe',
    'Saudi Arabia',
    'Senegal',
    'Serbia',
    'Sierra Leone',
    'Singapore',
    'Slovakia',
    'Slovenia',
    'South Africa',
    'South Sudan',
    'Spain',
    'Sri Lanka',
    'Sudan',
    'Sweden',
    'Switzerland',
    'Syria',
    'Taiwan',
    'Tajikistan',
    'Tanzania',
    'Thailand',
    'Timor-Leste',
    'Togo',
    'Tonga',
    'Trinidad and Tobago',
    'Tunisia',
    'Turkey',
    'Turkmenistan',
    'Uganda',
    'Ukraine',
    'United Arab Emirates',
    'United Kingdom',
    'United States of America',
    'Uruguay',
    'Uzbekistan',
    'Vanuatu',
    'Venezuela',
    'Viet Nam',
    'Yemen',
    'Zambia',
    'Zimbabwe']
Length: 195, dtype: str
```

Mean Years of Schooling vs HDI: Plot Mean Years of Schooling (X-axis) vs HDI Score (Y-axis) using a strip plot. Analyzes the education-HDI relationship.



Life Expectancy vs HDI: Plot Life Expectancy (X-axis) vs HDI Score (Y-axis). Reveals how longevity correlates with human development score.



Correlation Heatmap: Build a correlation matrix heatmap using Seaborn. Each cell value represents the correlation coefficient between two features. Use this to select the most relevant input columns for model training higher correlation with HDI indicates a stronger predictor.

```
corr_data = data[[
    "Life Expectancy at Birth (2021)",
    "Mean Years of Schooling (2021)",
    "Gross National Income Per Capita (2021)",
    "Human Development Index (2021)"
]]

corr_data = corr_data.apply(pd.to_numeric, errors="coerce")

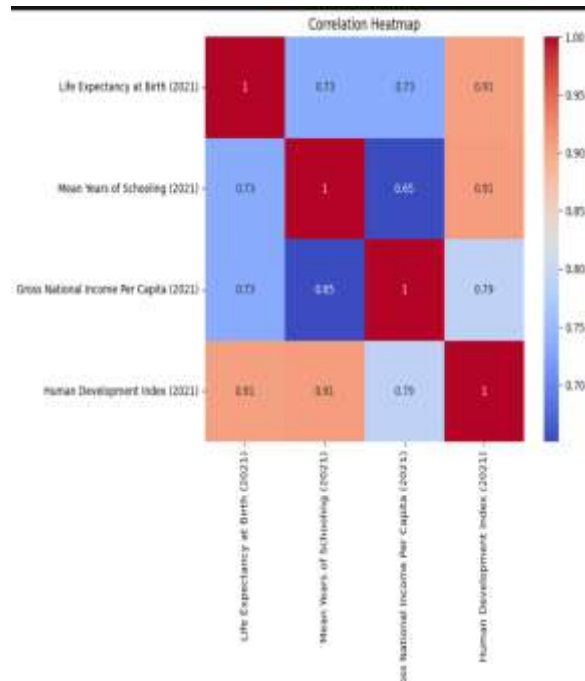
plt.figure(figsize=(8,6))

sns.heatmap(
    corr_data.corr(),
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")

plt.show()
```

✓ 1.2s



Epic 4: Data Preprocessing and Label Encoding

Story 1: Selecting Dependent and Independent Variables

Select features (independent variables) and the target (dependent variable) for the model.

Independent Variables (X): Inputs given to the model. Selected by their column index numbers e.g., index 2 for Country, index 5 for Life Expectancy, and so on. All independent variables are stored in X.

Dependent Variable (Y): The prediction target HDI score (column index 4). Stored in Y.

```
X = data[[
    "Country",
    "Life Expectancy at Birth (2021)",
    "Mean Years of Schooling (2021)",
    "Gross National Income Per Capita (2021)"
]]

y = data["Human Development Index (2021)"]
```

Story 2: Checking and Handling Null Values

Find Null Values:

Use `isnull().sum()` to count null values in each selected input column.

`X.isnull().sum()`

```
X.isnull().sum()
✓ 0.0s
```

Country	0
Life Expectancy at Birth (2021)	0
Mean Years of Schooling (2021)	0
Gross National Income Per Capita (2021)	0

dtype: int64

Fill Null Values in X:

Fill null values in the independent variable columns using the column mean.

`X = X.fillna(X.mean())`

```
X = X.fillna(X.mean(numeric_only=True))
✓ 0.0s
```

Epic 5: Dividing the Model into Train and Test Data

Story 1: Train and Test Data

Splitting data into training and testing sets is an important step in machine learning because it helps evaluate how well the model performs on unseen data. The training dataset is used to train the model by allowing it to learn patterns, relationships, and dependencies between the input features and target output. During this phase, the model adjusts its internal parameters to improve prediction accuracy and overall performance.

The testing dataset is used after training to evaluate the model's performance on new and unseen data. This helps determine whether the model can generalize effectively to real-world scenarios instead of only memorizing the training data. Performance metrics such as accuracy, precision, recall, and F1-score can be calculated using the testing dataset to measure the effectiveness and reliability of the model.

Use sklearn's `train_test_split`:

```
x_train, x_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
✓ 0.0s
```

(156, 4)
 (39, 4)
 (156,)
 (39,)

Epic 6: Fitting the Model

Story 1: Fit the Linear Regression Model

Import and instantiate the Linear Regression model to build a predictive machine learning model. The model is trained using the prepared training dataset so that it can learn the relationship between the input features and the target variable.

```
sample = pd.DataFrame(
    [[50,72.0,13.0,15000]],
    columns=[
        "Country",
        "Life Expectancy at Birth (2021)",
        "Mean Years of Schooling (2021)",
        "Gross National Income Per Capita (2021)"
    ]
)

prediction = reg.predict(sample)

print(prediction)
✓ 0.0s
[0.82399667]
```

Testing the test field form the split:

```
print(y_test.values)
✓ 0.0s
[0.699 0.795 0.477 0.607 0.887 0.939 0.498 0.661 0.758 0.747 0.921 0.812
 0.959 0.916 0.918 0.585 0.795 0.639 0.713 0.739 0.512 0.717 0.483 0.509
 nan 0.428 0.596 0.777 0.962 0.935 0.832 0.535 0.79 0.685 0.535 0.394
 0.78 0.577 0.621]
```

Story 2: Predicting the Results

Generate HDI Predictions: Run the model on the test set and print the predicted HDI values.

`print(y_pred)`

Calculate R-Squared Value: R-squared (R^2) measures the proportion of variance in the dependent variable explained by the independent variables. Test with Fewer Values: Test the model with a reduced input set to validate predictions for individual data points. Inspect `y_test` Values: Print the actual `y_test` values (ground truth HDI scores) as an array for comparison. Inspect `y_pred` Values: Print the predicted `y_pred` values. These should be very close to the `y_test` values, confirming the model's accuracy.

```
y_pred = reg.predict(x_test)

print(y_pred)
✓ 0.0s
[0.50196351 0.55978478 0.71969255 0.48289047 0.68564878 0.68620107
 0.92532069 0.79433554 0.92609473 0.45757254 0.87372211 0.48660253
 0.83068047 0.9316934 0.70316095 0.76876549 0.79740054 0.52898715
 0.40324057 0.90870551 0.98186562 0.79443198 0.55786796 0.52474372
 0.7497533 0.76628506 0.59790252 0.79456497 0.52201227 0.84172117
 0.74619545 0.86411746 0.81684506 0.80055459 0.75471091 0.61341427
 0.76432376 0.74095317 0.77367839]
```

```
score = r2_score(y_test, y_pred)

print("R² Score =", score)
✓ 0.0s
R² Score = 0.959955030014567
```

Epic 7: Saving the Model

Story 1: Saving the Model

Saving the model in this format allows us to reuse it whenever needed without having to retrain it from scratch, which saves both time and computational resources. In the deployed Flask web application, the saved model can be loaded directly and used to make predictions on new user inputs.

Pickle is a built-in Python module that enables the serialization and deserialization of Python objects. Serialization is the process of converting an object into a byte stream that can be stored on disk, while deserialization converts the stored byte stream back into a usable Python object. It ensures consistency in predictions, and allows the trained model to be integrated seamlessly into production applications.

```
✓ pickle.dump(reg, open("../Flask/HDI.pkl", "wb")) ...  
Model Saved Successfully
```

Epic 8: Building the Flask Web Application

Story 1: Writing the Flask Backend

Import the required libraries to build the web application. Flask is used as the web framework to create and manage the application routes, while Pickle is used to load the pre-trained machine learning model stored in a .pkl file. NumPy is imported to handle numerical input data required for prediction.

Next, initialize the Flask application and load the saved Human Development Index (HDI) prediction model. This allows the application to use the trained model for generating predictions based on user input.

```
from flask import Flask, render_template, request  
import pandas as pd  
import pickle  
import os  
  
app = Flask(__name__)  
  
model = pickle.load(open("HDI.pkl", "rb"))  
data = pd.read_csv("../Dataset/HDI.csv")  
  
# Country List  
countries = sorted(data["Country"].unique())  
  
# -----  
# Home Page  
# -----  
@app.route("/")  
def home():  
    return render_template("home.html")  
  
# -----  
# Prediction Page  
# -----  
@app.route("/predictpage")  
def predictpage():  
    return render_template(  
        "indexnew.html",  
        countries=countries  
    )
```

```
@app.route("/predict", methods=["POST"])
def predict():

    try:

        country = request.form["country"]

        life = float(request.form["life"])

        school = float(request.form["school"])

        gni = float(request.form["gni"])

        # Get encoded country value directly from dataset
        country_code = int(
            data.loc[data["Country"] == country].index[0]
        )

        sample = pd.DataFrame(
            [[country_code, life, school, gni]],
            columns=[
                "Country",
                "Life Expectancy at Birth (2021)",
                "Mean Years of Schooling (2021)",
                "Gross National Income Per Capita (2021)"
            ]
        )

        prediction = model.predict(sample)[0]

        # HDI Category
        if prediction >= 0.800:
            message = "Very High Human Development"

        elif prediction >= 0.700:
            message = "High Human Development"

        elif prediction >= 0.550:
            message = "Medium Human Development"

        else:
            message = "Low Human Development"
```

Story 2: Running the Application

Showcasing Predictions on the UI

The web application consists of three pages:

Home Page (home.html):

The **Home page code** is created using HTML and CSS to provide an attractive welcome interface for the application. It contains the project title, a brief description, and a navigation button that redirects users to the prediction page. This page serves as the starting point of the application.

```
task / templates / home.html / html
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6 <meta charset="UTF-8">
7 <meta name="viewport" content="width=device-width, initial-scale=1.0">
8
9 <title>Human Development Index Predictor</title>
10
11 <style>
12
13 *{
14     margin:0;
15     padding:0;
16     box-sizing:border-box;
17     font-family:Arial, Helvetica, sans-serif;
18 }
19
20 html,body{
21
22     width:100%;
23     height:100%;
24     overflow:hidden;
25
26 }
27
28 body{
29
30     background:url("{ url_for('static', filename='world.jpg') }");
31     background-size:cover;
32     background-position:center;
33     background-repeat:no-repeat;
34
35 }
36
37 .overlay{
38
39     width:100%;
40     height:100vh;
41
42     background: rgba(0,0,0,0.45);
```



```
39     width:100%;
40     height:100vh;
41
42     background: linear-gradient(to right, transparent 49%, #000000 49% 51%, #000000 51% 53%, transparent 53%);
43
44     display:flex;
45     justify-content:center;
46     align-items:center;
47
48 }
49
50 .container{
51
52     width:900px;
53
54     background: linear-gradient(to right, transparent 49%, #000000 49% 51%, #000000 51% 53%, transparent 53%);
55
56     border-radius:20px;
57
58     padding:50px;
59
60     box-shadow:0px 0px 25px #000000;
61
62     text-align:center;
63
64 }
65
66 h1{
67
68     color: #ff0000;
69
70     font-size:55px;
71
72     margin-bottom:20px;
73
74 }
75
76 h3{
77
78     color: #ff0000;
79
80     font-size:26px;
81
82     margin-bottom:35px;
83
84 }
85
86 p{
87
88     color: #ffffff;
89
90     font-size:22px;
91
92     line-height:38px;
93
94     text-align:justify;
95
96     margin-bottom:40px;
```

```
28 </style>
29
30 </head>
31
32 <body>
33
34 <div class="overlay">
35
36 <div class="container">
37
38 <h1>Human Development Index</h1>
39
40 <h3>Machine Learning Based Prediction System</h3>
41
42 <p>
43
44 The Human Development Index (HDI) is a statistical measure developed by the United Nations
45 to evaluate the overall development of countries based on three major dimensions:
46 Life Expectancy, Education, and Gross National Income (GNI) per capita.
47
48 This application uses a Machine Learning model to predict the Human Development Index
49 based on user-provided values. Select a country, enter the required indicators,
50 and instantly obtain the predicted HDI score.
51
52 </p>
53
54 <a href="/predictpage" class="button">
55
56 Predict HDI
57
58 </a>
59
60 </div>
61
62 </div>
63
64 </body>
65
66 </html>
```

Prediction Page (indexnew.html):

The **Index page code** contains an HTML form with input fields for the Human Development indicators, such as life expectancy, expected years of schooling, mean years of schooling, and Gross National Income per capita. When the user clicks the **Predict** button, the entered values are sent to the Flask backend using an HTTP POST request for prediction.

```
<!DOCTYPE html>
<html lang="en">

<head>

  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Human Development Index Predictor</title>

  <style>

    *{
      margin:0;
      padding:0;
      box-sizing:border-box;
      font-family:Arial, Helvetica, sans-serif;
    }

    body{

      background:url('{{ url_for('static', filename='world.jpg') }}');
      background-size:cover;
      background-position:center;
      background-repeat:no-repeat;

      height:100vh;

      display:flex;
      justify-content:center;
      align-items:center;

    }

    .overlay{

      width:100%;
      height:100%;

      background: rgba(0,0,0,.55);

      display:flex;
      justify-content:center;
      align-items:center;

    }

  </style>

</head>

<body>

  <div class="overlay">

    <div>

      <h1>Human Development Index Predictor</h1>

      <div>

        <div>Life Expectancy</div>
        <input type="text" value="75.5" />

        <div>Expected Years of Schooling</div>
        <input type="text" value="12.4" />

        <div>Mean Years of Schooling</div>
        <input type="text" value="8.5" />

        <div>Gross National Income per capita</div>
        <input type="text" value="12000" />

      </div>

      <div>

        <button>Predict</button>

      </div>

    </div>

  </div>

</body>

</html>
```

```

47
48   .container{
49
50     width:650px;
51
52     background: rgba(0,0,0,.80);
53
54     padding:40px;
55
56     border-radius:20px;
57
58     box-shadow:0px 0px 25px 0px black;
59
60   }
61
62   h1{
63
64     color: red;
65
66     text-align:center;
67
68     margin-bottom:35px;
69
70     font-size:42px;
71
72   }
73
74   label{
75
76     color: white;
77
78     font-size:18px;
79
80     font-weight:bold;
81
82     display:block;
83
84     margin-top:15px;
85
86     margin-bottom:8px;
87
88   }
89

```

```

113     color: black;
114
115   }
116
117   input::placeholder{
118
119     color: #ddd;
120
121   }
122
123   button{
124
125     width:100%;
126
127     margin-top:35px;
128
129     padding:16px;
130
131     border:none;
132
133     border-radius:35px;
134
135     font-size:22px;
136
137     font-weight:bold;
138
139     color: white;
140
141     cursor:pointer;
142
143     background:linear-gradient(to right, #36D1DC, #5B86E5);
144
145     transition:.3s;
146
147   }
148
149   button:hover{
150
151     transform:scale(1.03);
152
153   }
154
155 </style>
156

```

```

163 <div class="container">
164
165 <h1>Human Development Index</h1>
166
167 <form action="/predict" method="POST">
168
169 <label>Select Country</label>
170
171 <select name="country" required>
172
173 <option value="">-- Select Country --</option>
174
175 {% for country in countries %}
176
177 <option value="{{ country }}">{{ country }}</option>
178
179 {% endfor %}
180
181 </select>
182
183 <label>Life Expectancy at Birth (2021)</label>
184
185 <input
186 type="number"
187 step="0.1"
188 name="life"
189 placeholder="Enter Life Expectancy"
190 required>
191
192 <label>Mean Years of Schooling (2021)</label>
193
194 <input
195 type="number"
196 step="0.1"
197 name="school"
198 placeholder="Enter Mean Years of Schooling"
199 required>
200
201 <label>Gross National Income Per Capita (2021)</label>
202
203 <input
204 type="number"
205 step="0.1"
206 name="gni"
207 placeholder="Enter GNI Per Capita"
208 required>
209
210 <button type="submit">

```

Result Page (Result.html):

The result code uses the trained Linear Regression model to predict the Human Development Index based on the user-provided input values. The predicted HDI score is generated using the predict() function and is then classified into the appropriate Human Development category (Very High, High, Medium, or Low). Finally, the prediction result is displayed through the Flask web application for the user.

```

1  <!DOCTYPE html>
2  <html>
3
4  <head>
5
6      <title>HDI Result</title>
7
8      <style>
9
10         *{
11             margin:0;
12             padding:0;
13             box-sizing:border-box;
14             font-family:Arial, Helvetica, sans-serif;
15         }
16
17         body{
18
19             background:url('{{ url_for('static', filename='result.jpg') }}');
20             background-size:cover;
21             background-position:center;
22             background-repeat:no-repeat;
23
24             height:100vh;
25
26             display:flex;
27             justify-content:center;
28             align-items:center;
29
30         }
31

```

```

        .container{
            width:900px;
            background:#000;
            border-radius:20px;
            padding:60px;
            text-align:center;
            box-shadow:0 0 25px black;
        }
        h1{
            color:red;
            font-size:58px;
            margin-bottom:40px;
        }
        .country{
            color:white;
            font-size:32px;
            font-weight:bold;
            margin-bottom:40px;
        }
        .score{
            color:#00d9ff;
            font-size:80px;
            font-weight:bold;
            margin-bottom:40px;
        }
    
```

```

        .result{
            color:#00ff66;
            font-size:42px;
            font-weight:bold;
            margin-bottom:50px;
        }
    }

    .btn{
        display:inline-block;
        width:300px;
        height:65px;
        line-height:65px;
        text-decoration:none;
        color:white;
        font-size:30px;
        font-weight:bold;
        border-radius:40px;
        background:linear-gradient(to right,#4facfe,#b84cff);
    }
}

</style>
</head>
<body>
<div class="container">
    <h1>Human Development Index</h1>
    <div class="country">
        Country : {{ country }}

    <div class="country">
        Country : {{ country }}
    </div>
    <div class="score">
        {{ prediction }}
    </div>
    <div class="result">
        {{ message }}
    </div>
    <a href="/predictpage" class="btn">
        Predict Again
    </a>
</div>
</body>
</html>

```


Web Applications:

Home Page:

The Home Page serves as the entry point of the HDI Prediction System. It provides a brief introduction to the project and allows users to navigate to the prediction page. The interface is designed to be simple and user-friendly, making it easy for users to access the application's features.



Prediction Page:

The **Index Page** contains the prediction form where users enter the required Human Development indicators, such as life expectancy, years of schooling, and Gross National Income per capita. Once the input values are submitted, the data is sent to the Flask backend, which processes the request and forwards it to the trained machine learning model for prediction.



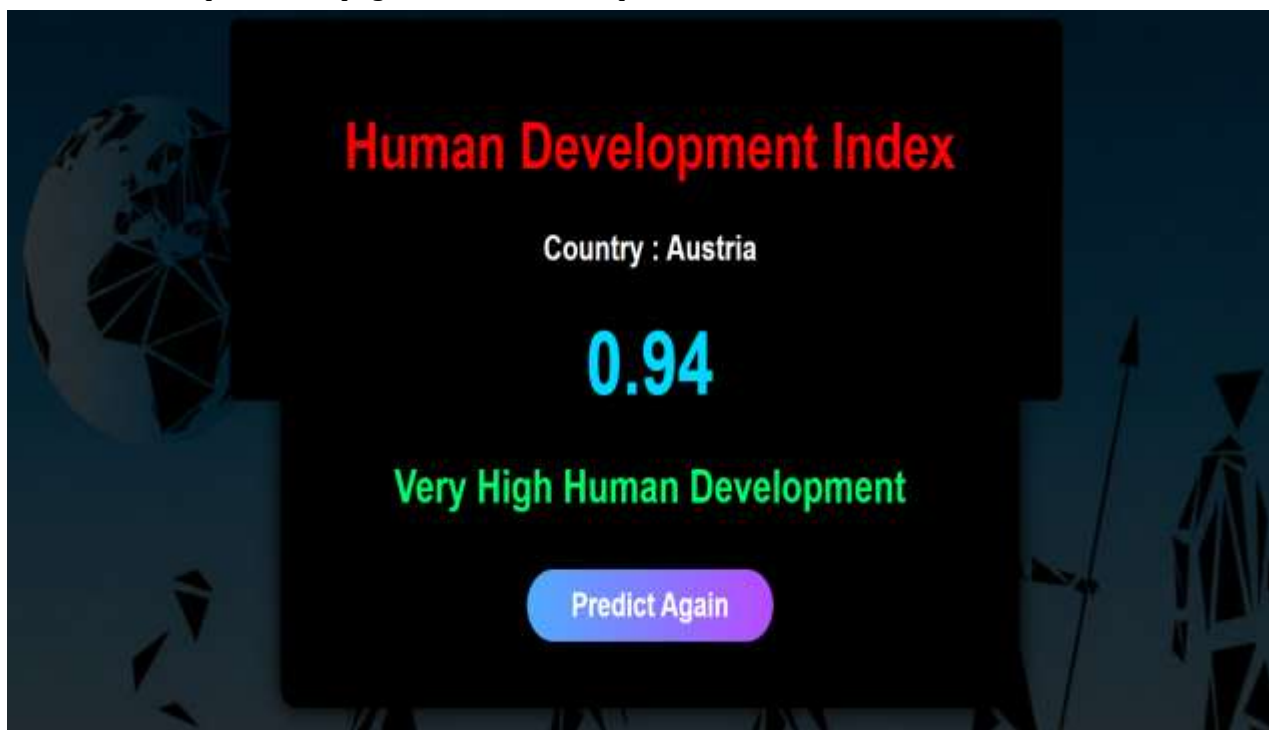
With Input:



The image shows a web interface for the Human Development Index (HDI) calculator. It features a dark blue background with a faint world map and binary code. The title 'Human Development Index' is in red. Below it, there are four input fields: 'Select Country' (a dropdown menu showing 'Austria'), 'Life Expectancy at Birth (2021)' (a text input with '82.5'), 'Mean Years of Schooling (2021)' (a text input with '12.5'), and 'Gross National Income Per Capita (2021)' (a text input with '55000'). At the bottom, there is a large blue button labeled 'Predict HDI'.

Result Page:

The Result Page displays the predicted Human Development Index (HDI) score generated by the Linear Regression model. It also shows the corresponding Human Development category, such as Very High, High, Medium, or Low. The page provides the prediction in a clear format and allows users to return to the prediction page to make another prediction.



The image shows the result page of the HDI calculator. It features a dark blue background with a faint world map and binary code. The title 'Human Development Index' is in red. Below it, the text 'Country : Austria' is displayed. The predicted HDI score '0.94' is shown in large, bold, light blue text. Below the score, the text 'Very High Human Development' is displayed in green. At the bottom, there is a large blue button labeled 'Predict Again'.

Conclusion:

The Human Development Index (HDI) Prediction System successfully demonstrates the application of Machine Learning to predict the level of human development using key socio-economic indicators. The project utilizes a Linear Regression model trained on the HDI dataset to generate accurate predictions based on factors such as Life Expectancy at Birth, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) per Capita. Proper data preprocessing, exploratory data analysis, feature selection, model training, and performance evaluation were carried out to ensure reliable prediction results.

To make the system accessible and user-friendly, a Flask web application was developed, enabling users to enter development indicators and instantly obtain the predicted HDI score along with its corresponding Human Development category. The project also follows a structured workflow that includes environment setup, data preprocessing, model development, model serialization using Pickle, and deployment through a web interface. This demonstrates the complete machine learning lifecycle from data collection to real-time prediction.

Overall, the project provides a simple, efficient, and practical solution for predicting the Human Development Index and showcases the integration of Python, Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, and Flask in building a real-world machine learning application. The developed system can assist researchers, students, policymakers, and government organizations in analyzing development indicators and supporting data-driven decision-making. In the future, the project can be enhanced by incorporating larger and more recent datasets, experimenting with advanced machine learning algorithms, and adding interactive dashboards and analytical reports to further improve prediction accuracy and user experience.